(2 Marks)

1. Explain PageRank Iteration Using MapReduce.

PageRank is an algorithm used to measure the importance of web pages based on incoming links. It is implemented using MapReduce to process large web graphs efficiently in a distributed system. The computation is done iteratively until the values converge.

- In **Map phase**, each page distributes its rank equally to outgoing links.
- Mapper emits rank contributions along with link structure.
- In **Reduce phase**, all incoming contributions are summed.
- New PageRank is calculated using the damping factor formula.
- The process is **iterative** and continues until convergence.
- MapReduce enables **parallel and scalable processing** of large data.

2. Explain Hubs and Authorities.

Hubs and Authorities is a link analysis algorithm used in web structure mining to rank web pages based on their link relationships. It assigns two scores to each page: **hub score** and **authority score**.

- A **Hub** is a page that links to many good authority pages (acts like a directory or collection page).
- An **Authority** is a page that contains useful information and is linked by many good hubs.
- Good hubs point to good authorities, and good authorities are linked by good hubs.
- Hub score depends on the authority scores of the pages it links to.
- Authority score depends on the hub scores of pages linking to it.
- The algorithm works iteratively and updates both scores until convergence.
- It is used in **search engines and web ranking systems** to improve relevance of results.
- Helps in identifying **important and trusted web pages** in a network.

3. Demonstrate Collaborative Filtering approach with example.

Collaborative Filtering is a recommendation technique used in data mining where recommendations are made based on the preferences and behavior of similar users. It assumes that users who agreed in the past will agree in the future.

- It is widely used in **recommendation systems** like Netflix, Amazon, and YouTube.
- It does not require item content; it only uses user-item interaction data (ratings, clicks, etc.).
- The basic idea is to find users with similar interests and recommend items liked by them.

Types of Collaborative Filtering

- **User-based filtering**: Finds similar users and recommends what they liked.
- **Item-based filtering**: Finds similar items and recommends items similar to what user liked.

User	Movie A	Movie B	Movie C
U1	5	4	?
U2	5	5	4
U3	1	2	5

- U1 and U2 have similar preferences.
- U2 liked Movie C (rating 4).
- So, **Movie C is recommended to U1**.

4. Explain HITS Algorithm.

HITS algorithm is a link analysis algorithm used to rank web pages based on their link structure. It identifies two types of important pages: **Hubs** and **Authorities**. It is mainly used in web structure mining to improve search results based on link relationships between pages.

- It assigns two scores to each page: **hub score** and **authority score**.
- A **hub** is a page that links to many good authority pages.
- An **authority** is a page that is linked by many good hub pages.
- Good hubs point to good authorities, and good authorities are linked by good hubs.
- The algorithm works iteratively and updates both scores until convergence.
- It is commonly used in **search engines** for ranking web pages.
- It helps in identifying relevant and trusted information sources on the web.

5. Define link Spam with examples.

Link spam refers to the manipulation of search engine ranking algorithms by creating artificial, irrelevant, or low-quality links to a website in order to increase its PageRank or visibility. It is a type of black-hat SEO technique that tries to deceive search engines and reduce the quality of search results.

- It involves creating **fake or irrelevant backlinks** to improve ranking.
- It increases artificial popularity of a website.
- It violates search engine policies and may lead to penalties or de-indexing.
- It reduces the reliability of search engine results.

Examples of Link Spam

- Creating multiple fake websites that all link to a target website.
- Adding irrelevant links in blog comments or forums just to increase backlinks.
- Using link farms where many low-quality sites link to each other.
- Hidden links placed in webpages (invisible to users but visible to search engines).

6. Explain Content-Based Recommendations.

Content-Based Recommendation is a technique used in recommendation systems where items are recommended to a user based on the similarity between item features and the user's past preferences. It focuses on the content or attributes of items rather than other users' behavior.

- It recommends items similar to those the user has liked or interacted with in the past.
- It uses **item features** such as keywords, category, genre, or description.
- A user profile is created based on previously liked items.
- Recommendations are made by matching item features with the user profile.
- It does not depend on other users' ratings (unlike collaborative filtering).

Example

- If a user watches **action movies**, the system will recommend other action movies like *Fast & Furious* or *John Wick*.
- If a user reads articles on **technology**, it will suggest similar tech-related articles.

7. Describe Counting triangles using Map-Reduce.

Counting triangles means finding the number of three nodes in a graph where each node is connected to the other two. It is an important measure in social network analysis and graph mining.

- In the **Map phase**, the mapper emits adjacency lists for each node or edge pairs.
- In the **Shuffle and Sort phase**, data is grouped by nodes to build adjacency information.
- In the **Reduce phase**, common neighbors between connected node pairs are identified to form triangles.
- Each detected triangle is counted, but duplicates occur due to multiple paths.
- Final count is corrected by dividing by 3 (or using proper normalization).
- MapReduce enables **parallel processing**, making triangle counting efficient for large graphs.
- It is widely used to analyze **network connectivity and clustering behavior**.

8. Explain the structure of the web with a dead end.

The web is represented as a **directed graph** where web pages are nodes and hyperlinks are directed edges. This structure shows how pages are connected and how information flows from one page to another through links. It is widely used in search engine ranking algorithms like PageRank to analyze importance of web pages. A **dead end** is a page that has incoming links but no outgoing links.

- A dead end page has **no outgoing links** to other pages.
- It is also called a **sink node** in the web graph.
- Dead ends absorb PageRank but do not distribute it further.
- This leads to **rank loss** during PageRank computation.
- It affects the stability and accuracy of ranking results.
- It is handled using **random teleportation (damping factor)**.
- This ensures proper distribution of rank and stable computation.

9. List down Big Data Analytics Applications and explain any 2 in detail.

Applications:

- Healthcare – Used to improve patient care and predict diseases.
- Banking and Finance – Used for fraud detection and risk management.
- Retail and E-commerce – Used for recommendation systems and customer analysis.
- Social Media – Used to analyze user behavior and trends.
- Transportation – Used for traffic prediction and route optimization.
- Manufacturing – Used to improve production efficiency and quality control.
- Education – Used to analyze student performance and learning patterns.
- Fraud Detection – Used to identify suspicious activities in real time.

1. Healthcare : Used to analyze patient data, predict diseases, and improve treatment. It helps in early diagnosis, better patient care, and reducing healthcare costs.

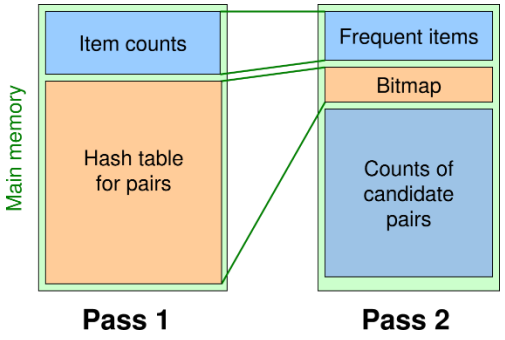
2. Banking and Finance : Used for fraud detection, risk analysis, and credit scoring. It helps banks identify suspicious transactions, improve security, and support better financial decisions.

(5 Marks)

10. Design Handling Larger Datasets in Main Memory Basic Algorithm of Park, Chen, and Yu.

Design Handling Larger Datasets in Main Memory – Park, Chen, and Yu Algorithm

The Park, Chen, and Yu algorithm is used for mining frequent itemsets when the dataset is too large to fit in main memory. It improves efficiency by using hashing and bitmap techniques to reduce memory usage and computation cost.



Explanation:

- The algorithm works completely in main memory using different data structures.
- Item counts store frequency of individual items during scanning.
- A hash table is used to store and count item pairs efficiently.
- Frequent items are selected based on minimum support.
- A bitmap is used for compact representation of frequent items.
- Candidate pair counts help in finding frequent item pairs.
- It reduces repeated database scanning and improves performance.

Working Steps:

- First scan identifies frequent items and updates pair counts using hashing.
- In-memory structures avoid multiple database scans.
- Bitmap helps in fast filtering of itemsets.
- Final frequent itemsets are generated after support checking.
- Only valid candidate itemsets are kept for next processing.

11. Describe CURE Algorithm with example

Definition: CURE is a hierarchical clustering algorithm used to handle large datasets and outliers. Unlike traditional methods that use only centroid, CURE represents each cluster using multiple representative points that are well scattered.

Key Idea:

- Each cluster is represented by a fixed number of well-scattered points.
- These points are then moved toward the centroid by a shrink factor.
- This helps in capturing shape and size of non-spherical clusters and reduces effect of outliers.

Working Steps:

- Start with each data point as a separate cluster.
- Select well-scattered representative points for each cluster.
- Shrink these points toward the cluster centroid by a fixed fraction (e.g., 0.2).
- Merge the two closest clusters based on the distance between their representative points.

• Repeat until required number of clusters is formed.

Example:
Assume data points are:
A(1,1), B(2,1), C(2,2), D(8,8), E(9,8), F(8,9)
• Initially, each point is its own cluster.
• CURE selects representative points (e.g., A, B, C form one cluster; D, E, F form another).
• After shrinking toward centroids:
- Cluster 1 becomes centered near (1.7, 1.3)
- Cluster 2 becomes centered near (8.3, 8.3)
  • These two clusters are far apart, so they are not merged further.
  • Final result: two clusters → one around (1,1) region and one around (8,8) region.

Advantages:
• Handles non-spherical clusters well
• Robust to outliers
• Works better than single-centroid methods

12. Explain Querying on Windows with DGIM algorithm.

DGIM (Datar–Gionis–Indyk–Motwani) algorithm is used to estimate the number of 1's in the last **N elements (sliding window)** of a binary data stream using very less memory.

Querying on Windows with DGIM Algorithm

- DGIM is used to answer queries like: "How many 1's are present in the last *k* elements of a stream?"
- It works on a **sliding window model**, where only the most recent N bits are considered.
- The stream is divided into **buckets**, where each bucket represents a group of 1's.
- Each bucket stores:
  - Size (number of 1's it represents: 1, 2, 4, 8, …)
  - Timestamp of the most recent bit in that bucket.
- Buckets are maintained such that:
  - No more than **2 buckets of the same size** exist.
  - Older buckets are merged into larger buckets when needed.

Query Processing Steps:

- To answer a query for last *k* bits:
  - Start from the most recent bucket.
  - Include all buckets fully inside the window.
  - For the oldest partially included bucket, take **half of its size (approximation)**.
  - Ignore buckets completely outside the window.

Result:

- Final answer is the **sum of all included bucket sizes (with approximation for last bucket)**.

Key Idea:

- Provides **approximate answer instead of exact count**.
- Uses **O(log N) memory instead of O(N)**.
- Suitable for **large data streams** where storing full data is not possible.

DGIM Example :

Binary stream: **1 0 1 1 0 1 1**

Find 1's in last **5 bits** using DGIM:

- Buckets formed: (sizes 1, 2, 2)
- Consider last 5 bits only

Counting:

- Last bucket (1) → include = 1
- Next bucket (2) → include = 2
- Oldest bucket partially in window → take half = 1

Final Answer:

Total 1's ≈ 1 + 2 + 1 = 4 (approximate)

13. Explain Flajolet-Martin Algorithm with an example.

- The Flajolet–Martin algorithm is used to estimate the **number of distinct elements (cardinality)** in a large data stream.
- It is a **probabilistic algorithm**, so it gives an **approximate answer** using very little memory.
- It is widely used in **big data and streaming systems**.
- It works on the idea of using **hash functions and binary representation** of data elements.

Key Idea:

- Each element in the stream is hashed using a hash function.
- The hash value is written in binary form.
- We observe the **position of the rightmost 1-bit (trailing zeros)**.
- The more distinct elements, the higher the number of trailing zeros observed.
- FM algorithm keeps track of the **maximum number of trailing zeros** found.

Steps of Algorithm:

- Apply a hash function to each element in the stream.
- Convert hash output into binary.
- For each value, count number of **trailing zeros**.
- Maintain a variable **R = maximum trailing zeros found**.
- Estimate distinct elements as:
  2^R

Example:

Consider a data stream: **A, B, C, D**

First, each element is hashed and converted into binary:

- A → 101100 → trailing zeros = 2
- B → 110000 → trailing zeros = 4
- C → 101010 → trailing zeros = 1
- D → 100000 → trailing zeros = 5

Now, find maximum trailing zeros:

- R = max(2, 4, 1, 5) = **5**

Estimation:

- Estimated number of distinct elements = **2^R = 2^5 = 32**

14. Construct Clustering with MapReduce Classification Algorithms (SVM Classifier)

Construct Clustering with MapReduce Classification Algorithms (SVM Classifier)

- MapReduce is a distributed programming model used to process **large datasets in parallel**.
- It works in two main phases: **Map phase and Reduce phase**.
- SVM (Support Vector Machine) is a **supervised learning algorithm** used for classification by finding an optimal separating hyperplane.
- In big data environments, SVM is implemented using MapReduce to improve scalability and speed.

MapReduce-based SVM Construction

1. Map Phase:

- The dataset is divided into multiple chunks and distributed across mappers.
- Each mapper trains SVM on its local data subset.
- It generates **partial models or support vectors**.
- Output: *(weight vectors, support vectors, intermediate classification results)*.

2. Shuffle and Sort Phase:

- Intermediate results from all mappers are grouped together.
- Data is organized so that all partial models are sent to the appropriate reducer.

3. Reduce Phase:

- Reducers combine all partial SVM models.
- It merges support vectors and updates global hyperplane parameters.
- Final output is a **single optimized SVM classifier model**.

Clustering with MapReduce (Concept Link):

- Although SVM is a classification method, MapReduce is also used for clustering tasks (like K-means).
- In clustering, mappers assign data points to clusters, and reducers recompute centroids.
- This concept helps in understanding how **MapReduce supports both clustering and classification** in big data.

Advantages:

- Efficient processing of **large-scale datasets**.
- Parallel training reduces computation time.
- Scalable for distributed systems.

15. Describe Filtering Streams: The Bloom Filter Counting

A **Counting Bloom Filter** is an advanced version of the standard Bloom Filter used for **stream data filtering and membership testing**, especially when deletion of elements is required. It is widely used in **data stream processing** to efficiently check whether an element is present in a large dataset.

Basic Idea

- It extends the Bloom Filter by replacing a bit array with a **counter array**.
- It allows **insertions and deletions** of elements in data streams.
- It still provides **probabilistic membership testing** (may give false positives but no false negatives in normal operation).

Structure

- A **counter array (C[0...m-1])** instead of a bit array
- **k independent hash functions**
- Each counter stores how many elements map to that position

Working

- When an element arrives in the stream, it is passed through **k hash functions**.
- Each hash function maps the element to a position in the counter array.
- The corresponding counters are **incremented by 1**.
- For **query (membership check)**:
  - If all k positions have **count > 0**, the element is considered **possibly present**.
  - If any position is **0**, the element is **definitely not present**.
- For **deletion**:
  - The same k positions are identified and their counters are **decremented**.

Algorithm Steps

1. Initialize a counter array of size m with all zeros.
2. Choose k hash functions.
3. For each incoming stream element:
   - Compute k hash values
   - Increment corresponding counters
4. For query:
   - Check all k positions
   - Decide membership based on counters
5. For deletion:
   - Decrease counters at k positions

Advantages

- Uses **very low memory** compared to storing actual data
- Supports **dynamic updates (insert + delete)**
- Fast processing suitable for **real-time data streams**
- Efficient for **large-scale datasets**

Limitations

- May produce **false positives**
- Counter overflow can occur if stream is very large
- Requires careful selection of **hash functions and array size**

Applications

- Network traffic filtering
- Spam detection systems
- Duplicate detection in data streams
- Database query optimization
- Web caching systems